

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский Авиационный Институт»
(Национальный Исследовательский Университет)

Институт: №8 «Информационные технологии и прикладная
математика»
Кафедра: 806 «Вычислительная математика и программирование»

Курсовая работа
по курсу «Вычислительные системы»
I семестр
Задание 4
«Процедуры и функции в качестве параметров»

Группа:	М8О-107Б-20
Студент:	Сысоев М.А.
Преподаватель:	Найдёнов И.Е.
Оценка:	
Дата:	

Москва, 2020

Постановка задачи

Составить программу на языке Си с процедурами решения трансцендентных алгебраических уравнений различными численными методами (итераций, Ньютона и половинного деления — дихотомия). Нелинейные уравнения оформить как параметры-функции, разрешив относительно неизвестной величины в случае необходимости. Применить каждую процедуру к решению двух уравнений, заданных двумя строками таблицы, начиная с варианта с заданным номером.

Вариант 13:

Уравнение:

$$x \cdot \operatorname{tg} x - \frac{1}{3} = 0$$

Отрезок: $[0.2, 1]$;

Базовый метод: Ньютона;

Приближённое значение корня: 0.5472;

Вариант 14:

Уравнение:

$$\operatorname{tg} \frac{x}{2} - \operatorname{ctg} \frac{x}{2} + x = 0$$

Отрезок: $[1, 2]$;

Базовый метод: дихотомии;

Приближённое значение корня: 1.0769;

Теоретическая часть

Метод половинного деления — простейший численный метод для решения нелинейных уравнений вида $f(x)=0$. Предполагается только непрерывность функции $f(x)$. Для начала итераций необходимо знать отрезок $[x_L, x_R]$ значений x , на концах которого функция принимает значения противоположных знаков. Это можно проверить так:

$$f(x_L) \cdot f(x_R) < 0$$

Из непрерывности следует, что на отрезке существует хотя бы один корень уравнения. Далее нужно найти значение x_M середины отрезка

$$x_M = x_L + x_R / 2$$

Вычислим значение функции $f(x_M)$ в середине отрезка. Если значения функции в середине отрезка и на левой границе разные

$$f(x_M) \cdot f(x_L) < 0,$$

то нужно переместить правую границу в середину отрезка, иначе левую границу в середину отрезка. Затем нужно повторить алгоритм начиная с вычисления значения x_M . Алгоритм заканчивается тогда, когда $f(x_M)=0$ либо $x_L=x_R$

Метод итераций — довольно простой численный метод решения уравнений. Метод основан на принципе сжимающего отображения, который применительно к численным методам в общем виде так же может называться методом простой итерации. Идея состоит в замене исходного уравнения $f(x)=0$ на эквивалентное ему $x=\varphi(x)$. При чём должно выполняться условие сходимости $|\varphi^{(1)}(x)| < 1$ на всём отрезке $[a, b]$. Итерации начинаются со значения x_M середины отрезка. Однако $\varphi(x)$ может выбрано неоднозначно. Сохраняет корни уравнения такое преобразование:

$$\varphi(x) = x - \lambda_0 \cdot f(x)$$

Здесь λ_0 — постоянная, которая не зависит от количества шагов. В данном случае мы возьмём

$$\lambda_0 = 1/f'(x_M),$$

что приводит к простому методу одной касательной и имеет условие сходимости

$\lambda_0 * f'(x) > 0$. Тогда итерационный процесс выглядит так:

$$x_{k+1} = x_k - \lambda_0 * f(x_k)$$

Условием окончания итераций является достижение нужной точности между предыдущим и следующим значением.

Метод Ньютона — итерационный численный метод нахождения корня заданной функции, который является частным случаем метода итераций. А именно за λ_0 берётся значение производной в каждой новой точке. Тогда итерационный процесс имеет вид

$$x_{k+1} = x_k - f(x_k) / f'(x_k)$$

Условие окончания итераций и начальное значение абсолютно такие же, как и в методе итераций. Условие сходимости метода можно записать как

$$|f(x) * f''(x)| < (f'(x))^2$$

Описание алгоритма

Рассмотрим алгоритм решения. Сперва нужно найти машинное эпсилон, на котором будет основываться точность вычисления. Это можно сделать просто деля 1 на 2. Сложность этого действия: $O(1)$

Опишем каждую из функций решения уравнений разными методами. Для метода дихотомии достаточно просто выполнять итерации, пока $x_R - x_L > \epsilon$. Корень с помощью такого метода рассчитывается примерно за

$$O(\log_2((R-L) * 10^k)) \sim O(k * \log_2(10 * (R-L))).$$

Все функции, их производные и сжимающие отображения зададим в виде функций-параметров. Алгоритмы для решения уравнений методом итерации и Ньютона реализованы так же, как и было описано в теоретической части. Невозможно оценить сложность, так как она зависит от самой функции. Будем сохранять все корни и сразу же изводить, не затрачивая память на новые переменные.

Использованные в программе структуры данных

Название структуры	Переменные в структуре	Смысл структуры
root_x	int steps double x	steps – то самое N, число итераций, затраченное на вычисление корня. x – то самое x_0 , искомое значение корня

Использованные в программе переменные

Название переменной	Тип переменной	Смысл переменной
k	int	То самое число K, используемое для вычисления точности. Так же обозначает, что вывод должен быть с точность до K знаков после запятой
e0	double	То самое машинное эpsilon. В случае с double $\varepsilon = 2.20 * 10^{-16}$
acc	double	Точность вычислений. Именно с этой переменной мы будем сравнивать ответы
x0	root	Значение корня, которое будут возвращать функции после вычисления.

Использованные в программе функции

Название функции	Тип возвращаемой переменной	Смысл функции
square	double	Возвращает квадрат числа
f13	double	То самое $f_1(x)$
f14	double	То самое $f_2(x)$
squeeze_f13	double	То самое $\varphi_1(x)$
squeeze_f14	double	То самое $\varphi_2(x)$
d_dx_f13	double	То самое $f_1^{(1)}(x)$
d_dx_f14	double	То самое $f_2^{(1)}(x)$
solve_binary_search	root	Решение уравнения методом половинного деления. Принимает f – функцию-параметр $f(x)$, $l - x_L$, $r - x_R$, k и асс
solve_iteration	root	Решение уравнения методом итерации. Принимает f – функцию-параметр $\varphi(x)$, $x_0 - x_M$, k и асс
solve_newton	root	Решение уравнения методом Ньютона. Принимает f – функцию-параметр $f(x)$, $derivative_f$ – функцию-параметр $f^{(1)}(x)$, $x_0 - x_M$, k и асс

Исходный код программы:

```
#include <math.h>
```

```
#include <stdio.h>
```

```
typedef struct root_x root;
```

```
struct root_x
```

```
{
```

```
double x;
```

```
int steps;
```

```
};
```

```
double square(double x)
```

```
{
```

```
return x * x;
```

```
}
```

```
root solve_binary_search(double f(double), double l, double r, int k, double  
acc)
```

```
{
```

```
int step = 0;
```

```
double m;
```

```
while (r - l > acc) {
```

```
step++;
```

```
m = (l + r) / 2.0;
```

```

if (f(m) * f(l) < 0) {
    r = m;
}
else {
    l = m;
}
}
root ans = { m, step };
return ans;
}

```

```

root solve_iteration(double f(double), double x0, int k, double acc)
{
    int step = 0;
    double cur = x0;
    double prev = cur + 1;
    while (fabs(cur - prev) > acc) {
        prev = cur;
        cur = f(prev);
        step++;
    }
    root ans = { cur, step };
    return ans;
}

```



```
}
```

```
root solve_newton(double f(double), double derivative_f(double), double x0,  
int k, double acc)
```

```
{
```

```
int step = 0;
```

```
double cur = x0;
```

```
double prev = cur + 1;
```

```
while (fabs(cur - prev) > acc) {
```

```
prev = cur;
```

```
cur = prev - f(prev) / derivative_f(prev);
```

```
step++;
```

```
}
```

```
root ans = { cur, step };
```

```
return ans;
```

```
}
```

```
double f13(double x) {
```

```
return x * tan(x) - 1.0 / 3.0;
```

```
}
```

```
double squeeze_f13(double x) {
```

```
return x - f13(x) / 1.564962;
```

```
}
```

```
double d_dx_f13(double x) {  
    return tan(x) + x / square(cos(x));  
}
```

```
double f14(double x) {  
    return tan(x / 2.0) - 1 / tan(x / 2.0) + x;  
}
```

```
double squeeze_f14(double x) {  
    return x - f14(x) / 3.010057;  
}
```

```
double d_dx_f14(double x) {  
    return 1.0 / (2.0 * square(cos(x / 2.0))) + 1 / (2.0 * square(sin(x / 2.0))) +  
    1;  
}
```

```
int main() {  
    int k;  
    scanf_s("%d", &k);  
    double e0 = 1.0;  
    while ((1.0 + e0 / 2.0) > 1.0) {  
        e0 = e0 / 2.0;
```

```

}

double acc = e0 * pow(10, 16 - k);

root x0;

printf("Machine epsilon equals %.8e\n", e0);
printf("_____ \n");

printf("Answer for  $x \cdot \tan(x) - 1/3 = 0$ \n");

x0 = solve_binary_search(f13, 0.2, 1.0, k, acc);

printf("%.5f | %d\n", k, x0.x, x0.steps);


x0 = solve_iteration(squeeze_f13, (0.2 + 1.0) / 2.0, k, acc);

printf("%.5f | %d\n", k, x0.x, x0.steps);

x0 = solve_newton(f13, d_dx_f13, (0.2 + 1.0) / 2.0, k, acc);

printf("%.5f | %d\n", k, x0.x, x0.steps);

printf("Answer for  $\tan(x/2) - \cot(x/2) + x = 0$ \n");

x0 = solve_binary_search(f14, 1.0, 2.0, k, acc);

printf("%.5f | %d\n", k, x0.x, x0.steps);

x0 = solve_iteration(squeeze_f14, (1.0 + 2.0) / 2.0, k, acc);

printf("%.5f | %d\n", k, x0.x, x0.steps);

x0 = solve_newton(f14, d_dx_f14, (1.0 + 2.0) / 2.0, k, acc);

printf("%.5f | %d\n", k, x0.x, x0.steps);

return 0;

}

```

Входные данные

Единственная строка содержит одно целое число K ($0 \leq K \leq 16$) — коэффициент для точности вычисления искомого корня.

Выходные данные

Программа должна вывести значение машинного эпсилон, а затем 6 строк. В каждой строке необходимо вывести искомое значение корня x_0 с точностью K знаков после запятой и N — число итераций. В первых трёх строках для первого уравнения, а в следующих трёх для другого. Следует выводить сначала значения, полученные методом дихотомии, потом методом итерации, затем методом Ньютона.

Протокол исполнения и тесты

Тест №1

Ввод:

3

Вывод:

Machine epsilon equals 2.22044605e-16

Answer for $x \cdot \operatorname{tg}(x) - 1/3 = 0$

0.548 | 9

0.547 | 3

0.547 | 3

Answer for $\operatorname{tg}(x/2) - \operatorname{ctg}(x/2) + x = 0$

1.076 | 9

1.077 | 4

1.077 | 3

Тест №2

Ввод:

6

Вывод:

Machine epsilon equals 2.22044605e-16

Answer for $x \cdot \operatorname{tg}(x) - 1/3 = 0$

0.547160 | 19

0.547161 | 6

0.547161 | 4

Answer for $\operatorname{tg}(x/2) - \operatorname{ctg}(x/2) + x = 0$

1.076876 | 19

1.076874 | 8

1.076874 | 4

Тест №3

Ввод:

9

Вывод:

Machine epsilon equals 2.22044605e-16

Answer for $x \cdot \operatorname{tg}(x) - 1/3 = 0$

0.547160758 | 29

0.547160758 | 9

0.547160757 | 4

Answer for $\operatorname{tg}(x/2) - \operatorname{ctg}(x/2) + x = 0$

1.076873986 | 29

1.076873987 | 12

1.076873986 | 5

Тест №4

Ввод:

12

Вывод:

Machine epsilon equals 2.22044605e-16

Answer for $x \cdot \operatorname{tg}(x) - 1/3 = 0$

0.547160757262 | 39

0.547160757260 | 13

0.547160757260 | 5

Answer for $\operatorname{tg}(x/2) - \operatorname{ctg}(x/2) + x = 0$

1.076873986311 | 39

1.076873986312 | 17

1.076873986312 | 5

Тест № 5

Ввод:

15

Вывод:

Machine epsilon equals 2.22044605e-16

Answer for $x \cdot \operatorname{tg}(x) - 1/3 = 0$

0.547160757260330 | 49

0.547160757260330 | 16

0.547160757260330 | 5

Answer for $\operatorname{tg}(x/2) - \operatorname{ctg}(x/2) + x = 0$

1.076873986311805 | 49

1.076873986311804 | 21

1.076873986311804 | 5

Вывод

В работе описаны идеи и принципы трёх численных методов: дихотомии, итераций и Ньютона. Проведены нужные вычисления для использования методов. Составлен алгоритм решения уравнений, на основе которого составлена программа на языке Си. Описан формат ввода и вывода, проведено тестирование программы, составлен протокол исполнения программы.

Работа была довольно трудной, так как не был изучен курс численных методов, из-за чего пришлось самостоятельно изучать сложный материал. Однако, это не делает её неинтересной. Скорее, даже, наоборот.

Список литературы

1. Метод бисекции – URL: https://ru.wikipedia.org/wiki/Метод_бисекции
2. Метод простой итерации – URL: https://ru.wikipedia.org/wiki/Метод_простой_итерации
3. Метод Ньютона – URL: https://ru.wikipedia.org/wiki/Метод_Ньютона